처음 배우는 WPF
4주 통합 교재

개발환경 안내와 16회 학습자료 · 총 14시간

한글 글꼴을 포함한 A4 읽기용 PDF입니다. 목차를 누르면 해당 수업으로 이동합니다. 코드 파일 링크는 원본 프로젝트와 함께 사용하세요. 긴 코드 줄은 페이지 폭에 맞춰 표시하며 실행에는 원본 소스를 사용합니다.

개발환경 및 프로젝트 구성 안내

1주차 1일차: WPF와 첫 번째 창 — 30분

1주차 2일차: XAML과 기본 레이아웃 — 30분

1주차 3일차: 입력과 버튼 이벤트 — 30분

1주차 4일차: 입력 → 실행 → 결과 완성 — 120분

2주차 1일차: DataGrid에 C# 목록 표시 — 30분

2주차 2일차: Binding과 DataContext — 30분

2주차 3일차: TwoWay와 속성 변경 알림 — 30분

2주차 4일차: ObservableCollection과 표 갱신 — 120분

3주차 1일차: Model·View·ViewModel의 역할 — 30분

3주차 2일차: ViewModel로 실행 동작 옮기기 — 30분

3주차 3일차: ICommand와 RelayCommand — 30분

3주차 4일차: Toolkit으로 MVVM 기본 구조 완성 — 120분

4주차 1일차: async/await로 기다리는 동안 화면 유지 — 30분

4주차 2일차: HttpClient로 고정 URL GET 호출 — 30분

4주차 3일차: 입력 URL과 기본 오류 처리 — 30분

4주차 4일차: 작은 WPF API 테스트 프로그램 완성 — 120분

개발환경 및 프로젝트 구성 안내

이 안내는 학습 시작 전 준비용입니다. 설치·다운로드 시간은 14시간 학습 시간에서 제외합니다. WPF는 Windows에서 실행하는 .NET UI 기술입니다. 이 교재는 .NET 10과 Visual Studio 2026을 기준으로 합니다.

1. 필요한 도구 준비

1. Visual Studio 다운로드에서 Visual Studio 2026을 설치합니다. 개인 학습에서는 사용 조건에 맞는 Community 에디션을 선택할 수 있습니다.
2. Visual Studio Installer의 **워크로드**에서 **.NET 데스크톱 개발**을 선택합니다.
3. 개별 구성 요소에서 .NET 10 SDK가 포함되어 있는지 확인하고 설치합니다.
4. Visual Studio를 실행합니다.

이미 설치되어 있다면 Visual Studio Installer에서 해당 Visual Studio의 **수정**을 눌러 워크로드를 확인합니다. SQL Server Management Studio(SSMS) 설치만으로는 WPF 개발환경이 준비되지 않습니다.

이 환경 선택은 Microsoft의 WPF 프로젝트 생성 안내의 Visual Studio 2026·.NET 데스크톱 개발·.NET 10 구성을 기준으로 합니다.

.NET Framework는 기존 Windows 프로그램에서 많이 사용하던 계열입니다. 이 교재는 최신 .NET 계열의 .NET 10을 사용하므로 프로젝트 생성 시 **WPF 애플리케이션(.NET Framework)** 대신 C# **WPF 애플리케이션 / WPF Application**을 선택합니다. 이 차이 외의 역사나 마이그레이션은 학습하지 않습니다.

2. 제공 프로젝트로 시작하기 — 권장

1. 파일 탐색기에서 저장소의 sample/WpfTestApp.sln을 더블클릭합니다.
2. 솔루션 탐색기에 WpfTestApp 프로젝트와 MainWindow.xaml이 보이는지 확인합니다.
3. F5를 누릅니다. 현재 제공 프로젝트는 4주차 완료본이므로 입력창, 실행 버튼, 결과 표가 표시됩니다. 첫 빌드에는 CommunityToolkit.Mvvm 8.4.2 패키지 복원이 필요합니다.
4. 기본 API URL을 그대로 두고 실행하면 게시글이 표에 표시됩니다. 인터넷 연결이 필요합니다.
5. 창을 닫아 실행을 종료합니다.

제공 상태는 **4주차 4일차 완료본(day16)**입니다. 1일차부터 따라가려면 실행을 종료하고 다음 순서로 바꿉니다. 단계 적용 후 1~2주차에는 외부 패키지가 필요 없습니다.

1. Visual Studio의 **보기 → 터미널**을 엽니다. 현재 위치를 저장소 루트(Tutorial_WPF)로 맞춥니다.
2. powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 1을 실행합니다. 교재 파일을 자동 백업하고 해당 단계의 소스와 프로젝트 설정을 함께 적용합니다.
3. Visual Studio가 파일 변경을 물으면 다시 불러옵니다.
4. F5로 실행하여 "첫 번째 WPF 창입니다."가 나타나는지 확인합니다.

날짜별 파일은 시작 코드가 아니라 **그날의 완료본**입니다. 2일차 수업은 1일차 완료 상태에서 시작합니다. stages 폴더 자체를 프로젝트 안에 넣지 않습니다. 스크립트 대신 파일 탐색기로 적용하려면 예제 안내의

전체 파일 복사 절차를 사용하세요.

1주차 완료 상태는 Stage 4, 2주차 완료는 Stage 8, 3주차 완료는 Stage 12, 최종 완료는 Stage 16입니다. 모든 수업은 sample/WpfTestApp 프로젝트에서 이어갑니다. 3주차 3일차부터 패키지 설치가 필요하며 해당 교재에 절차가 있습니다.

3. 직접 새 프로젝트를 만드는 경우

제공 프로젝트로 시작했다면 이 절차는 건너뛰어도 됩니다. 직접 만들더라도 학습에는 프로젝트 하나만 사용합니다.

1. Visual Studio 시작 화면에서 **새 프로젝트 만들기**를 선택합니다.
2. 검색창에 wpf를 입력하고 언어를 **C#**으로 선택합니다.
3. **WPF 애플리케이션 / WPF Application** 템플릿을 고릅니다.
4. 프로젝트 이름을 반드시 WpfTestApp으로 정합니다. 저장 위치는 제공 예제와 다른 빈 폴더로 선택합니다.
5. 프레임워크는 **.NET 10.0**을 선택합니다.
6. 생성 후 F5로 빈 창이 실행되는지 확인하고 닫습니다.
7. 1일차 교재에서 MainWindow 두 파일을 교체하며 시작합니다.

이름을 통일하면 XAML의 x:Class="WpfTestApp.MainWindow"와 C#의 namespace WpfTestApp이 바로 맞아떨어집니다. 프로젝트 이름과 네임스페이스를 바꾸는 방법은 이번 학습에서 생략합니다.

4. 지금 알아둘 프로젝트 파일

파일	쉬운 설명	이번 주에 할 일
WpfTestApp.csproj	대상 .NET과 WPF 사용 여부 등 프로젝트 설정	열어 보기만 함
App.xaml	프로그램 전체의 시작 설정. StartupUri로 첫 창 지정	수정하지 않음
App.xaml.cs	프로그램 전체에 대한 C# 코드	수정하지 않음
MainWindow.xaml	화면의 구성과 배치	수정
MainWindow.xaml.cs	그 화면에 연결된 C# 동작 코드	수정
WpfTestApp.sln	Visual Studio에서 프로젝트를 묶어 여는 파일	더블클릭하여 열기

실행 흐름은 "프로그램 시작 → App의 StartupUri 확인 → MainWindow 생성 → InitializeComponent로 화면 초기화 → 사용자의 입력과 클릭 대기"입니다. 콘솔 앱처럼 우리가 작성한 Main 메서드부터 보이지 않아도 실행 시작 부분은 빌드 과정에서 준비됩니다.

bin, obj 폴더는 빌드가 생성합니다. 교재에서 수정하지 않습니다.

5. 실행에 문제가 있을 때 확인할 최소 항목

증상	확인할 내용
WPF 템플릿이나 .NET 10이 보이지 않음	Visual Studio Installer에서 .NET 데스크톱 개발과 .NET 10 SDK 확인
.NET 10 대상을 지원하지 않는다는 오류	Visual Studio 2026과 .NET 10 SDK로 열었는지 확인
InitializeComponent나 컨트롤 이름을 찾을 수 없음	MainWindow 두 파일의 날짜, x:Class·namespace, x:Name 철자를 확인하고 솔루션 다시 빌드
버튼 이벤트 메서드를 찾을 수 없음	XAML의 Click 값과 C# 메서드 이름이 같은지 확인
실행 버튼이 다른 프로젝트를 가리킴	솔루션 탐색기에서 WpfTestApp 우클릭 → 시작 프로젝트로 설정
XAML 미리보기가 열리지 않음	XAML 텍스트 편집 화면에서 진행하고 빌드·F5 실행 결과 확인

XAML을 변경하기 전에는 실행 중인 창을 닫으세요. 교재는 핫 리로드 설정에 의존하지 않습니다.

터미널 확인은 선택 사항입니다. 저장소 루트에서 다음 명령을 사용할 수 있습니다.

```
dotnet --list-sdks
dotnet build .₩sample₩WpfTestApp.sln
dotnet run --project .₩sample₩WpfTestApp₩WpfTestApp.csproj
```

준비가 끝나면 1주차 1일차로 이동합니다.

1주차 1일차: WPF와 첫 번째 창 — 30분

1. 학습 목표 — 2분

- WPF가 C#으로 Windows 화면 프로그램을 만드는 기술임을 이해합니다.
- MainWindow.xaml과 MainWindow.xaml.cs를 구분합니다.
- Visual Studio에서 첫 창을 실행합니다.

개발환경 설치는 사전 준비 안내를 따라 미리 완료합니다.

2. 핵심 개념 설명 — 8분

콘솔 앱에서는 Console.WriteLine으로 결과를 출력합니다. WPF 앱에서는 창 안의 텍스트나 표에 결과를 표시하고, 사용자의 버튼 클릭에 반응합니다. C#의 변수, 조건문, 메서드는 그대로 사용합니다. 달라지는 부분은 화면을 만들고 화면의 사건에 반응하는 방식입니다.

WPF는 Windows 전용 화면 프레임워크입니다. 화면 요소의 배치와 데이터 연결 기능을 제공합니다. 글자와 도형은 벡터 기반으로 그려져 화면 배율에 대응할 수 있습니다. 지금은 "Windows 창을 C#과 연결해서 만드는 기술"로 이해하면 충분합니다. Microsoft WPF 개요

파일	역할
App.xaml	StartupUri에 지정한 MainWindow.xaml을 첫 창으로 엽니다.
MainWindow.xaml	어떤 화면 요소를 배치할지 적습니다.
MainWindow.xaml.cs	그 화면의 C# 코드를 적습니다. Code-behind라고 부릅니다.
WpfTestApp.csproj	.NET 대상과 WPF 사용 설정을 보관합니다.

XAML은 XML 형태의 화면 작성 문법입니다. 여는 태그와 닫는 태그로 요소를 감싸고, 요소의 설정은 속성으로 적습니다. <TextBlock Text="안녕하세요" />는 "TextBlock을 하나 만들고 Text 속성을 설정하라"는 뜻입니다. TextBlock은 화면에 글자를 보여 주는 요소입니다. 끝의 />는 자식 요소가 없을 때 태그를 한 번에 닫는 표기입니다.

C# 파일에서 알아둘 연결은 세 가지입니다.

- x:Class="WpfTestApp.MainWindow": XAML과 C#의 MainWindow 클래스를 연결합니다.
- partial: 빌드 과정에서 생성되는 부분과 우리가 작성한 부분이 하나의 클래스를 이룹니다.
- InitializeComponent(): XAML에 정의한 화면 요소와 설정을 초기화합니다. 직접 구현하거나 삭제하지 않습니다.

xmlns 두 줄은 WPF 요소와 x: 문법을 알아보기 위한 이름 공간 선언입니다. 인터넷 주소 모양이지만 실행할 때 그 주소로 접속해 화면을 다운로드하는 것은 아닙니다. 이번 주에는 그대로 둡니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

아래는 오늘 완료 시점의 두 파일 전체입니다. 같은 이름의 파일 내용을 각각 교체합니다. 두 파일을 하나로 합치지 않습니다.

MainWindow.xaml

```
<Window x:Class="WpfTestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF 텍스트 테스트" Height="360" Width="640"
        MinHeight="300" MinWidth="420">
    <Grid Margin="16">
        <TextBlock Text="첫 번째 WPF 창입니다." />
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

Window는 창, Grid는 자식을 배치하는 영역입니다. 오늘은 Grid 안에 글자 하나만 둡니다. Title은 창 제목이고, Width와 Height는 창 크기입니다. Margin은 바깥 여백입니다. 크기 값은 WPF의 화면 배율을 고려하는 단위이며, 오늘은 예제 값을 그대로 사용합니다.

C# 파일은 화면을 초기화하는 생성자만 있습니다. 아직 클릭 동작은 없습니다.

4. Visual Studio에서 따라 하는 실습 — 12분

1. **2분:** 제공 솔루션 sample/WpfTestApp.sln을 엽니다. 환경 안내에서 day01 파일로 바꿨다면 그대로 사용합니다. 직접 생성한 프로젝트도 이름은 WpfTestApp이어야 합니다.
2. **4분:** 솔루션 탐색기에서 MainWindow.xaml을 더블클릭하고 XAML 텍스트 편집 영역을 엽니다. 위 XAML 전체로 교체합니다. MainWindow.xaml 왼쪽 펼침 화살표를 눌러 MainWindow.xaml.cs를 열고 C# 코드 전체로 교체합니다.
3. **2분:** 모두 저장한 뒤 F5를 누릅니다. 제목이 "WPF 텍스트 테스트"이고 창 안에 "첫 번째 WPF 창입니다."가 보이는지 확인합니다.
4. **3분:** 창을 닫습니다. XAML의 Text 값만 "C#과 화면이 연결되었습니다."로 바꿔 저장하고 F5로 다시 실행합니다. 글자가 바뀌는 것을 확인한 뒤 창을 닫고 Text를 원래 값으로 되돌립니다.

5. **1분:** App.xaml을 열어 StartupUri가 MainWindow.xaml인지 확인합니다. App.xaml은 수정하지 않습니다.

제공 프로젝트의 csproj에는 TargetFramework가 net10.0-windows, UseWPF가 true로 설정되어 있습니다. 한 번 확인만 하고 그대로 둡니다.

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- WPF를 사용해도 프로그램의 동작은 익숙한 C#으로 작성합니다.
- XAML은 화면 요소와 속성을 표현하고, Code-behind는 그 화면과 연결된 C# 코드입니다.
- MainWindow.xaml과 MainWindow.xaml.cs는 연결된 하나의 창을 구성합니다.
- 실행은 App → MainWindow 생성 → InitializeComponent → 창 표시 순서로 이어집니다.

완료본은 day01 폴더에 있습니다. 다음은 2일차: XAML과 기본 레이아웃입니다.

1주차 2일차: XAML과 기본 레이아웃 — 30분

1. 학습 목표 — 2분

- Grid와 StackPanel로 화면을 배치합니다.
- TextBox, Button, TextBlock의 역할을 구분합니다.
- 입력 영역과 결과 영역이 있는 화면을 만듭니다.

시작 상태는 1일차 완료본입니다.

2. 핵심 개념 설명 — 7분

화면의 작은 구성 요소를 컨트롤이라고 부릅니다. 입력받는 TextBox, 눌러서 실행하는 Button, 글자를 보여 주는 TextBlock부터 사용합니다.

요소	역할	이번 화면에서의 예
TextBox	사용자가 글자를 편집	테스트할 텍스트 입력
Button	사용자의 실행 동작을 받음	실행 버튼
TextBlock	읽을 글자를 표시	안내와 결과

배치는 두 종류만 익힙니다.

Grid는 행과 열로 나누는 배치 영역입니다. 이번에는 창 전체를 안내, 입력, 결과의 세 행으로 나눕니다. 입력 행 안에 작은 Grid를 넣어 입력창과 버튼을 두 열로 나눕니다. Grid는 데이터를 표로 표시하는 DataGrid와 다른 역할입니다.

- Auto: 내용이 필요한 만큼 공간을 사용합니다.
- *: 다른 부분이 사용하고 남은 공간을 사용합니다.
- Grid.Row="1": 두 번째 행에 배치합니다. 행과 열 번호는 0부터 시작하고, 생략하면 0입니다.
- Grid.Column="1": 두 번째 열에 배치합니다.

StackPanel은 자식을 순서대로 쌓는 배치 영역입니다. 기본 방향은 세로입니다. 결과 제목과 결과 문구를 위아래로 쌓는 데 사용합니다.

기본 창은 직접 자식 하나를 받습니다. 그래서 Grid를 하나 넣고 그 안에 여러 요소를 배치합니다. 좌표를 일일이 계산하지 않아도 창 크기에 맞춰 배치가 조정됩니다.

Margin="0,0,8,0"은 왼쪽·위·오른쪽·아래 순서의 바깥 여백입니다. Padding="16,6"은 버튼 내용의 좌우·상하 안쪽 여백입니다. TextWrapping="Wrap"은 긴 글자를 줄바꿈합니다. 꾸미는 방법을 외우기보다 요소가 겹치지 않고 읽히는지만 확인하세요.

3. 짧고 이해하기 쉬운 코드 예제 — 6분

아래는 오늘 완료 시점의 두 파일 전체입니다. 같은 이름의 파일 내용을 각각 교체합니다. 두 파일을 하나로 합치지 않습니다.

MainWindow.xaml

```
<Window x:Class="WpfTestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF 텍스트 테스트" Height="360" Width="640"
        MinHeight="300" MinWidth="420">
    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <TextBlock Text="테스트할 텍스트를 입력하세요."
                   Margin="0,0,0,8" />

        <Grid Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>

            <TextBox x:Name="InputTextBox"
                     Text="안녕하세요, WPF"
                     VerticalContentAlignment="Center"
                     Margin="0,0,8,0" />
            <Button Grid.Column="1" Content="실행"
                    Padding="16,6" />
        </Grid>

        <StackPanel Grid.Row="2" Margin="0,16,0,0">
            <TextBlock Text="결과" Margin="0,0,0,8" />
            <TextBlock x:Name="ResultTextBlock"
                       Text="아직 실행하지 않았습니다."
                       TextWrapping="Wrap" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

오늘 C# 파일은 1일차와 같습니다. x:Name은 다음 시간에 C#에서 해당 요소를 찾을 이름입니다. TextBox와 TextBlock은 Text를, Button은 Content를 화면에 표시합니다. 아직 Button에 Click 연결이 없으므로 버튼을 눌러도 결과는 바뀌지 않습니다.

4. Visual Studio에서 따라 하는 실습 — 12분

1. **2분:** 실행 중인 창을 닫고 MainWindow.xaml을 엽니다. 위 코드에서 세 행과 입력 행의 두 열을 먼저 찾습니다.
2. **5분:** XAML 전체를 교체하고 저장합니다. C# 파일은 그대로 둡니다. 1일차와 다른 날짜의 코드가 섞였다면 위 C# 전체로 맞춥니다.
3. **2분:** F5로 실행합니다. 안내 아래에 입력창과 실행 버튼, 그 아래에 결과 문구가 나타나는지 확인합니다. 입력창의 기본 글자를 지우고 다른 글자를 입력합니다.
4. **2분:** 창을 넓혀 봅니다. 입력창이 넓어지고 버튼은 필요한 너비를 유지하는지 봅니다. 이것이 입력 행의 별표 열과 Auto 열이 하는 일입니다.
5. **1분:** 실행을 종료합니다. TextBox는 편집할 수 있고 결과 TextBlock은 직접 편집할 수 없다는 차이를 확인합니다.

코드가 길어 보여도 대부분은 배치입니다. 이번 시간에는 행 세 개, 입력 행의 열 두 개, 결과의 세로 쌓기만 구분하면 됩니다.

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- Grid는 행과 열로 나누고, StackPanel은 자식을 순서대로 쌓습니다.
- Auto는 필요한 크기, 별표는 남는 공간입니다.
- TextBox는 입력, Button은 실행, TextBlock은 표시를 담당합니다.
- XAML로 버튼을 배치하는 것과 클릭했을 때 할 일을 연결하는 것은 별개입니다.

완료본은 day02 폴더에 있습니다. 다음은 3일차: 입력과 버튼 이벤트입니다.

1주차 3일차: 입력과 버튼 이벤트 — 30분

1. 학습 목표 — 2분

- Click 이벤트와 C# 메서드를 연결합니다.
- C#에서 입력 컨트롤의 값을 읽고 결과 컨트롤에 씁니다.
- 버튼을 누를 때마다 동작하는 프로그램을 실행합니다.

시작 상태는 2일차 완료본입니다.

2. 핵심 개념 설명 — 6분

이벤트는 "어떤 일이 일어났다"는 알림입니다. 버튼의 Click 이벤트에 메서드를 연결하면 사용자가 버튼을 누를 때 WPF가 그 메서드를 호출합니다. 이 메서드를 이벤트 처리기라고 부릅니다.

콘솔 앱에서 입력문을 차례로 실행하던 방식과 달리, 창은 열린 상태로 사용자의 동작을 기다립니다. 클릭할 때마다 처리기가 다시 호출됩니다.

오늘의 연결은 다음과 같습니다.

```
버튼 클릭
  → XAML의 Click="RunButton_Click"
  → C#의 RunButton_Click 호출
  → InputTextBox.Text 읽기
  → ResultTextBlock.Text에 쓰기
```

InputTextBox와 ResultTextBlock은 XAML의 x:Name으로 지정한 이름입니다. 초기화가 끝나면 Code-behind에서 그 이름으로 화면 요소에 접근할 수 있습니다. TextBox에 입력된 글자는 Text라는 string 속성에 들어 있습니다.

object sender는 이벤트를 일으킨 객체, RoutedEventArgs e는 이벤트 관련 정보입니다. 오늘은 사용하지 않으므로 인자 두 개를 그대로 둡니다. 이벤트가 화면 안에서 전달되는 자세한 원리는 다루지 않습니다.

실무·면접 기초: Code-behind는 화면 객체를 직접 읽고 수정하기 편합니다. 데이터와 동작이 늘어날 때의 역할 분리는 바인딩과 MVVM을 배운 뒤 비교합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

2일차에서 바뀌는 핵심은 XAML의 Click="RunButton_Click"과 C#의 메서드 한 개입니다. 위치가 헷갈리지 않도록 두 파일 전체를 제공합니다.

아래는 오늘 완료 시점의 두 파일 전체입니다. 같은 이름의 파일 내용을 각각 교체합니다. 두 파일을 하나로 합치지 않습니다.

MainWindow.xaml

```
<Window x:Class="WpfTestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF 텍스트 테스트" Height="360" Width="640"
        MinHeight="300" MinWidth="420">
    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <TextBlock Text="테스트할 텍스트를 입력하세요."
                   Margin="0,0,0,8" />

        <Grid Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>

            <TextBox x:Name="InputTextBox"
                     Text="안녕하세요, WPF"
                     VerticalContentAlignment="Center"
                     Margin="0,0,8,0" />
            <Button Grid.Column="1" Content="실행"
                    Padding="16,6"
                    Click="RunButton_Click" />
        </Grid>

        <StackPanel Grid.Row="2" Margin="0,16,0,0">
            <TextBlock Text="결과" Margin="0,0,0,8" />
            <TextBlock x:Name="ResultTextBlock"
                       Text="아직 실행하지 않았습니다."
                       TextWrapping="Wrap" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
```

```
    private void RunButton_Click(object sender, RoutedEventArgs e)
    {
        ResultTextBlock.Text = InputTextBox.Text;
    }
}
```

ResultTextBlock.Text = InputTextBox.Text;는 실행되는 순간의 입력값을 읽어서 복사하는 일반적인 C# 대입문입니다. 자동으로 계속 연결되는 기능은 아닙니다. 버튼을 누른 뒤 입력창을 다시 편집해도 결과는 다음 클릭까지 그대로입니다.

4. Visual Studio에서 따라 하는 실습 — 14분

1. **2분:** 실행을 종료하고 MainWindow.xaml과 MainWindow.xaml.cs를 엽니다.
2. **5분:** 두 파일을 위 예제 전체로 교체하고 저장합니다. XAML의 Click 이름과 C# 메서드 이름이 정확히 같은지 확인합니다. 이벤트를 자동 생성하는 메뉴는 사용하지 않아도 됩니다.
3. **3분:** F5를 누릅니다. 기본 텍스트가 입력된 상태에서 실행 버튼을 누르면 결과에 같은 글자가 나타나는지 봅니다.
4. **2분:** 입력을 "두 번째 입력"으로 바꿉니다. 버튼을 누르기 전에는 기존 결과가 유지되고, 버튼을 누른 뒤에 바뀌는 것을 확인합니다.
5. **2분:** 창을 닫고 C#의 대입문을 다시 봅니다. 왼쪽은 결과를 쓸 대상, 오른쪽은 값을 읽을 대상입니다.

실행 버튼을 눌러도 반응하지 않으면 XAML에 Click이 들어 있는지 확인합니다. 빌드 오류가 나면 두 파일의 처리기 이름과 x:Name 철자를 확인합니다.

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- 이벤트 처리기는 사용자의 동작에 따라 WPF가 호출하는 C# 메서드입니다.
- XAML의 Click과 C#의 메서드 이름으로 연결합니다.
- x:Name이 있으면 Code-behind에서 해당 요소를 참조할 수 있습니다.
- 오늘은 클릭할 때만 값을 복사합니다. 데이터 바인딩은 아직 사용하지 않았습니다.

완료본은 day03 폴더에 있습니다. 다음은 4일차: 입력 → 실행 → 결과 완성입니다.

1주차 4일차: 입력 → 실행 → 결과 완성 — 120분

1. 학습 목표 — 5분

- 앞서 만든 입력창과 이벤트 처리기를 하나의 동작 흐름으로 이해합니다.
- 일반적인 C# 조건문을 WPF 이벤트 처리기에 적용합니다.
- 빈 입력 안내를 포함한 1주차 프로그램을 완성합니다.

시작 상태는 3일차 완료본입니다. 오늘 새로 추가하는 것은 입력값 확인과 상태 문구뿐입니다. 남은 시간은 같은 프로그램을 실행하고 디버거로 값을 따라가는 데 사용합니다.

2. 핵심 개념 설명 — 15분

버튼 클릭 후 처리할 순서를 먼저 말로 정해 봅니다.

```
현재 입력값 읽기
  → 앞뒤 공백 제거
  → 내용이 비어 있으면 안내하고 종료
  → 내용이 있으면 결과와 상태 표시
```

이 순서를 C#으로 적으면 됩니다. 화면 프로그램이라고 해서 조건문이나 string 처리 방법이 달라지지 않습니다.

- Trim(): 읽어 온 문자열의 앞뒤 공백을 제거합니다. TextBox 자체를 수정하는 것은 아닙니다.
- string.IsNullOrWhiteSpace(input): 입력이 없거나 공백뿐인지 확인합니다.
- return: 현재 이벤트 처리기만 끝냅니다. 창이나 프로그램 전체를 닫지 않습니다.

예를 들어 입력창이 " 테스트 "이면 결과는 "테스트"가 됩니다. 입력창에는 원래 글자가 남습니다. 입력창이 공백뿐이면 결과 영역에는 결과가 없다는 안내를, 상태 문구에는 입력 요청을 표시합니다.

결과와 상태는 역할이 다릅니다. 결과는 사용자가 얻으려는 값이고, 상태는 "표시 완료", "입력 필요"와 같은 진행 안내입니다. 다음 주에 결과가 표로 바뀌어도 상태 문구는 그대로 활용합니다.

XAML의 안내 문구만 바꾸는 일과 C#의 처리 순서를 바꾸는 일은 구분해서 생각합니다. UI(User Interface)는 사용자가 조작하거나 보는 화면 부분을 뜻합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

XAML에는 상태용 TextBlock 하나를 추가합니다. C#에는 입력값을 변수에 저장하고, 빈 입력이면 안내하는 조건문을 추가합니다.

아래는 오늘 완료 시점의 두 파일 전체입니다. 같은 이름의 파일 내용을 각각 교체합니다. 두 파일을 하나로 합치지 않습니다.

MainWindow.xaml

```
<Window x:Class="WpfTestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF 텍스트 테스트" Height="360" Width="640"
        MinHeight="300" MinWidth="420">
    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <TextBlock Text="테스트할 텍스트를 입력하세요."
                   Margin="0,0,0,8" />

        <Grid Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>

            <TextBox x:Name="InputTextBox"
                     Text="안녕하세요, WPF"
                     VerticalContentAlignment="Center"
                     Margin="0,0,8,0" />
            <Button Grid.Column="1" Content="실행"
                    Padding="16,6"
                    Click="RunButton_Click" />
        </Grid>

        <StackPanel Grid.Row="2" Margin="0,16,0,0">
            <TextBlock Text="결과" Margin="0,0,0,8" />
            <TextBlock x:Name="ResultTextBlock"
                       Text="아직 실행하지 않았습니다."
                       TextWrapping="Wrap" />
            <TextBlock x:Name="StatusTextBlock"
                       Text="입력 후 실행 버튼을 누르세요."
                       Margin="0,12,0,0" TextWrapping="Wrap" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void RunButton_Click(object sender, RoutedEventArgs e)
    {
        string input = InputTextBox.Text.Trim();

        if (string.IsNullOrWhiteSpace(input))
        {
            ResultTextBlock.Text = "표시할 결과가 없습니다.";
            StatusTextBlock.Text = "텍스트를 먼저 입력해 주세요.";
            return;
        }

        ResultTextBlock.Text = input;
        StatusTextBlock.Text = "입력한 텍스트를 표시했습니다.";
    }
}
```

처리기에서 화면 값을 직접 읽고 쓰는 부분은 세 컨트롤의 Text 속성입니다. 나머지는 익숙한 C# 문자열 처리와 조건문입니다. 줄마다 외우기보다 "읽기 → 확인 → 표시"를 기억하세요.

4. Visual Studio에서 따라 하는 실습 — 75분

파일 준비와 구현 — 25분

1. 실행을 종료하고 MainWindow 두 파일을 엽니다.
2. XAML 전체를 교체합니다. 아래쪽 StackPanel에 StatusTextBlock이 추가된 것을 확인합니다.
3. C# 전체를 교체합니다. 생성자는 유지하고 RunButton_Click의 내용이 확장된 것을 확인합니다.
4. 모두 저장하고 **빌드 → 솔루션 빌드**를 누릅니다. 오류가 있으면 오류 목록에서 첫 오류를 더블클릭합니다. 컨트롤 이름과 Click 처리기 이름부터 맞춥니다.
5. F5로 실행해 입력창과 버튼, 결과와 상태 문구가 보이는지 확인합니다.

입력에 따른 화면 동작 확인 — 20분

1. "WPF 테스트"를 입력하고 실행을 누릅니다. 결과는 "WPF 테스트", 상태는 "입력한 텍스트를 표시했습니다."가 됩니다.
2. 입력을 모두 지우고 실행합니다. 이전 결과 대신 "표시할 결과가 없습니다."가 나타나고 상태는 "텍스트를 먼저 입력해 주세요."가 됩니다.
3. 공백만 넣고 실행해 같은 안내가 나오는지 확인합니다.

4. 앞뒤에 공백이 있는 " 다시 시작 "을 입력하고 실행합니다. 결과는 "다시 시작"이고, 입력창의 앞뒤 공백은 남아 있습니다.
5. 긴 문장을 넣고 창 너비를 조절합니다. 결과가 다음 줄로 넘어가는 것을 봅니다. 화면을 꾸미기 위한 코드는 추가하지 않습니다.

C# 디버거로 실행 흐름 관찰 — 20분

1. 실행 중인 창을 닫습니다.
2. C#의 `string input = InputTextBox.Text.Trim();` 줄 왼쪽 여백을 클릭해 중단점을 표시합니다. 같은 줄에서 F9를 눌러도 됩니다.
3. F5로 실행하고 입력 후 버튼을 누릅니다. Visual Studio가 해당 줄에서 멈춥니다. 디버거가 멈춘 동안 앱이 반응하지 않는 것은 정상입니다.
4. F10을 한 번 눌러 대입문을 실행합니다. 그다음 input에 마우스를 올리거나 **디버그 → 창 → 지역**에서 값을 확인합니다. 노란 줄은 다음에 실행할 줄입니다.
5. F10으로 조건문과 결과 표시를 따라갑니다. F5를 누르면 계속 실행합니다.
6. 공백만 입력한 뒤 버튼을 다시 누릅니다. 이번에는 if 내부를 거쳐 return에서 처리기가 끝나는 것을 봅니다. F5로 계속 실행하면 창은 열린 채 유지됩니다.
7. 중단점을 다시 클릭하거나 F9로 제거하고 실행을 종료합니다.

최종 저장과 화면 확인 — 10분

1. 중단점이 제거된 상태에서 저장하고 F5로 다시 실행합니다.
2. 텍스트 입력 → 클릭 → 결과 표시까지 한 번 이어서 진행합니다.
3. MainWindow.xaml의 Click과 C# 메서드를 나란히 열고, 입력값을 읽는 줄과 결과를 쓰는 줄의 연결을 확인합니다.
4. 창을 닫습니다. 다음 주에는 이 프로젝트를 그대로 이어 씁니다.

시간이 부족하면 긴 문장의 창 너비 조절을 먼저 생략하고, 정상 입력과 빈 입력의 동작을 우선 확인합니다. 별도의 응용 프로그램은 만들지 않습니다.

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- WPF의 화면 요소도 C#에서 속성을 읽고 쓸 수 있는 객체입니다.
- XAML은 배치와 초기 속성, Code-behind는 클릭 후 동작을 담당했습니다.
- 입력값 읽기, 문자열 처리, 조건문은 일반적인 C# 코드입니다.
- return은 클릭 처리기를 끝내며, 다음 클릭에는 같은 메서드가 다시 호출됩니다.
- 현재는 화면을 직접 읽고 수정합니다. 다음 주에는 C# 데이터 속성과 화면을 바인딩으로 연결합니다.

실무·면접 기초: "WPF에서 XAML과 C#은 어떻게 연결되는가?"에는 x:Class로 같은 창 클래스를 연결하고, InitializeComponent로 화면을 초기화하며, Click으로 이벤트 처리기를 연결한다고 설명할 수 있으면 충분합니다.

완료본은 day04 폴더입니다. 지금 학습 상태에는 API 호출이나 데이터 표가 아직 없습니다. 다음은 2주차 1일차입니다. 현재 제공 프로젝트는 전체 과정 완료 상태이므로, 1주차 완료 상태가 필요하면 단계 적용 안내에 따라 Stage 4를 적용합니다.

2주차 1일차: DataGrid에 C# 목록 표시 — 30분

1. 학습 목표 — 2분

- DataGrid의 한 행이 C# 객체 하나에 대응함을 이해합니다.
- 고정 예제 두 개를 표로 표시합니다.

시작 상태는 1주차 완료본(day04)입니다. 입력창과 실행 버튼은 유지합니다.

2. 핵심 개념 설명 — 6분

TextBlock은 문장 하나를 보여 주는 데 적합합니다. 번호와 제목이 여러 줄이면 DataGrid를 사용합니다. MSSQL 결과 표를 떠올려도 좋지만, DataGrid가 데이터베이스에 연결되는 것은 아닙니다. 오늘은 메모리의 List<Post>를 보여 줍니다.

C# 데이터	화면
Post 객체 한 개	표의 행 한 개
Id 속성	번호 열의 값
Title 속성	제목 열의 값
List<Post>	표에 표시할 여러 행

ItemsSource는 "이 목록을 행으로 보여 주세요"라는 설정입니다. AutoGenerateColumns=False로 자동 열 생성을 끄고 번호·제목만 정합니다. IsReadOnly=True는 셀 편집을 막습니다. CanUserAddRows=False는 맨 아래 새 행 입력줄을 없앱니다.

열의 Binding은 각 행의 Post에서 속성값을 읽습니다. 오늘은 "Id 속성을 번호 열에 표시한다" 정도만 이해합니다. 바인딩의 기준 객체는 다음 시간에 다룹니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
namespace WpfTestApp;

public class Post
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
}
```

생성자에서 InitializeComponent 다음에 목록을 지정합니다.

```
PostsGrid.ItemsSource = new List<Post>
{
    new Post { Id = 1, Title = "첫 번째 예제 게시글" },
    new Post { Id = 2, Title = "두 번째 예제 게시글" }
};
```

DataGrid의 전체 배치는 XAML 링크를 사용합니다. 버튼은 입력값을 확인하며, 오늘의 표는 고정 예제이므로 버튼을 눌러도 행이 바뀌지 않습니다.

오늘 바뀌는 파일입니다. 링크에는 생략 없는 전체 코드가 있습니다.

파일	전체 코드
MainWindow.xaml	day05의 MainWindow.xaml
Post.cs	day05의 Post.cs
MainWindow.xaml.cs	day05의 MainWindow.xaml.cs

본문에는 핵심 부분만 보여 줍니다. 파일을 만들거나 교체할 때는 위 링크의 **전체 코드**를 사용하세요. App 파일은 바뀌지 않습니다.

4. Visual Studio에서 따라 하는 실습 — 14분

1. **2분:** Visual Studio에서 기존 WpfTestApp 솔루션을 엽니다. 1주차 창이 실행되는 상태를 확인하고 닫습니다.
2. **6분:** 프로젝트 우클릭 → 추가 → 클래스에서 Post.cs를 만들고 위 전체 코드를 넣습니다. MainWindow 두 파일도 오늘의 전체 코드로 교체합니다.
3. **3분:** 저장하고 F5를 누릅니다. 번호 1·2와 한국어 제목 두 개가 표에 보이는지 확인합니다.
4. **3분:** 입력을 바꾸고 실행 버튼을 누릅니다. 입력 결과 문구만 바뀌고 표의 두 행은 유지되는 것을 봅니다. 창을 닫습니다.

진도가 다르면 저장소 루트의 터미널에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 4로 시작 상태를 적용할 수 있습니다. 파일 복사가 어렵다면 같은 명령의 Stage를 5로 바꾸면 오늘 완료본이 적용됩니다. 실행 중인 앱을 먼저 닫고 Visual Studio에서 변경 파일을 다시 불러옵니다. 기존 교재 파일은 자동 백업됩니다. 단계 적용 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- DataGrid는 C# 목록을 여러 행으로 보여 줍니다.
- ItemsSource는 행으로 표시할 데이터 목록입니다.
- 한 행 안의 열은 그 Post의 public 속성을 읽습니다.
- 오늘은 MSSQL과 인터넷을 사용하지 않습니다.

완료본: day05. 다음: 2주차 2일차.

2주차 2일차: Binding과 DataContext — 30분

1. 학습 목표 — 2분

- 화면에 값을 직접 대입하는 코드와 Binding을 구분합니다.
- DataContext가 속성을 찾는 기준 객체임을 이해합니다.

시작 상태는 day05입니다.

2. 핵심 개념 설명 — 7분

1주차에는 ResultTextBlock.Text에 직접 값을 썼습니다. 바인딩은 XAML에 "이 객체의 Result 속성을 표시해 주세요"라고 연결을 선언하는 방법입니다.

DataContext는 그 객체를 지정합니다. 오늘은 MainWindow에 Result와 Posts 속성을 두고, 생성자에서 DataContext=this로 창 자신을 기준 객체로 지정합니다. this는 현재 MainWindow 객체를 뜻합니다. 자식 요소들은 보통 부모의 DataContext를 물려받습니다.

XAML	읽는 대상
Text="{Binding Result, Mode=OneWay}"	창의 Result 속성
ItemsSource="{Binding Posts, Mode=OneWay}"	창의 Posts 목록
열의 Binding="{Binding Title}"	해당 행 Post의 Title 속성

OneWay는 데이터에서 화면으로 값을 전달하는 방향입니다. "계속 변화를 자동 감지한다"는 뜻까지 포함하지는 않습니다. 첫 연결 때 값은 읽지만, 이후 C# 속성 변경을 화면에 알리는 장치가 필요합니다. 오늘 일부러 그 장치가 없는 상태를 관찰합니다. 공식 바인딩 설명

public 속성으로 선언해야 바인딩에서 값을 읽을 수 있습니다. Result와 result는 철자가 다르므로 구분합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
public string Result { get; set; } = "바인딩으로 읽은 첫 번째 결과입니다.";
public List<Post> Posts { get; } = new()
{
    new Post { Id = 1, Title = "첫 번째 예제 게시글" },
    new Post { Id = 2, Title = "두 번째 예제 게시글" }
};

public MainWindow()
{
    InitializeComponent();
    DataContext = this;
}
```

XAML에서 결과와 목록을 연결합니다.

```
<TextBlock Text="{Binding Result, Mode=OneWay}" />
<DataGrid ItemsSource="{Binding Posts, Mode=OneWay}" />
```

마지막 XAML 두 줄은 연결 부분을 보여 주는 짧은 예입니다. 실제 배치·열 정의는 전체 XAML 파일을 사용합니다. 오늘의 버튼은 C# Result를 변경하지만 변경 알림이 없어 결과 문구는 그대로 남습니다. 이는 오늘 관찰할 정상 동작입니다.

오늘 바뀌는 파일입니다. 링크에는 생략 없는 전체 코드가 있습니다.

파일	전체 코드
MainWindow.xaml	day06의 MainWindow.xaml
MainWindow.xaml.cs	day06의 MainWindow.xaml.cs

본문에는 핵심 부분만 보여 줍니다. 파일을 만들거나 교체할 때는 위 링크의 **전체 코드**를 사용하세요. App 파일은 바뀌지 않습니다.

4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** 창을 닫고 오늘 바뀌는 두 파일 전체를 교체합니다. Post.cs는 유지합니다.
2. **4분:** 생성자에서 DataContext=this를 찾고 Result·Posts가 MainWindow의 public 속성인지 확인합니다.
3. **3분:** F5로 실행합니다. "바인딩으로 읽은 첫 번째 결과입니다."와 두 행이 보이면 최초 데이터 읽기가 된 것입니다.
4. **4분:** 입력을 바꾸고 실행합니다. 아래 상태 문구는 바뀌지만 결과 문구는 그대로입니다. 처리기 첫 줄에 중단점을 걸어 한 줄 실행한 뒤 Result 값은 바뀌었는지 확인합니다. 중단점을 제거하고 창을 닫습니다.

진도가 다르면 저장소 루트의 터미널에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 5로 시작 상태를 적용할 수 있습니다. 파일 복사가 어렵다면 같은 명령의 Stage를 6로 바꾸면 오늘 완료본이 적용됩니다. 실행 중인 앱을 먼저 닫고 Visual Studio에서 변경 파일을 다시 불러옵니다. 기존 교재 파일은 자동 백업됩니다. 단계 적용 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- Binding은 데이터 속성과 화면 속성의 연결 선언입니다.
- DataContext는 속성 경로를 찾는 기준 객체입니다.
- 표의 열은 창이 아니라 해당 행 객체를 기준으로 값을 읽습니다.
- C# 값이 바뀌었다는 사실을 별도로 알려야 화면이 다시 읽습니다.

완료본: day06. 다음: 2주차 3일차.

2주차 3일차: TwoWay와 속성 변경 알림 — 30분

1. 학습 목표 — 2분

- 입력값을 C# Input 속성에 연결합니다.
- INotifyPropertyChanged로 결과와 상태 변경을 화면에 알립니다.

시작 상태는 day06입니다.

2. 핵심 개념 설명 — 7분

OneWay는 데이터 → 화면, TwoWay는 데이터 ↔ 화면입니다. 입력창은 사용자가 편집하므로 TwoWay로 연결합니다. 다만 방향과 전달 시점은 별개입니다.

TextBox.Text는 기본적으로 포커스를 잃을 때 데이터에 편집값을 반영합니다. 오늘은 UpdateSourceTrigger=PropertyChanged를 지정하여 타이핑할 때마다 Input에 반영합니다. TextBox 갱신 시점 공식 설명

반대 방향의 변경도 알릴 필요가 있습니다. C#에서 Result나 Status를 바꿀 때 INotifyPropertyChanged의 PropertyChanged 이벤트로 "이 속성이 바뀌었다"고 알립니다. 바인딩은 알림을 받고 값을 다시 읽습니다. 변경 알림 공식 설명

코드	하는 일
Input의 set	입력창에서 넘어온 값을 저장
Result의 private set	클래스 내부 처리 코드만 결과를 변경
Notify(nameof(Result))	Result가 바뀌었음을 알림

nameof(Result)는 속성 이름을 문자열로 얻습니다. event의 ?와 ?.Invoke는 등록된 대상이 없으면 호출을 건너뛰기 위한 C# 표기입니다. 이 반복 코드는 전체 파일에서 가져오고, "값 저장 다음에 알림"이라는 순서만 이해하세요.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
<TextBox Text="{Binding Input, Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}" />
```

Result 속성과 알림 메서드는 다음 역할을 합니다.

```
    private string result = "아직 실행하지 않았습니다.";
    public string Result
    {
        get => result;
        private set
        {
            result = value;
            Notify(nameof(Result));
        }
    }
```

```
    public event PropertyChangedEventHandler? PropertyChanged;

    private void Notify(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
```

실제 화면 배치는 전체 파일에 있습니다. Input과 Status에도 같은 방식의 알림이 들어 있습니다. 이벤트 처리기에서는 InputTextBox.Text 대신 Input을 읽고, ResultTextBlock.Text 대신 Result를 바꿉니다.

오늘 바뀌는 파일입니다. 링크에는 생략 없는 전체 코드가 있습니다.

파일	전체 코드
MainWindow.xaml	day07의 MainWindow.xaml
MainWindow.xaml.cs	day07의 MainWindow.xaml.cs

본문에는 핵심 부분만 보여 줍니다. 파일을 만들거나 교체할 때는 위 링크의 **전체 코드**를 사용하세요. App 파일은 바뀌지 않습니다.

4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** MainWindow 두 파일을 오늘의 전체 코드로 교체합니다. Post.cs는 바뀌지 않습니다.
2. **4분:** 클래스 선언의 INotifyPropertyChanged, 세 속성, Notify 메서드를 찾습니다. 실행 메서드가 화면 컨트롤 이름을 직접 사용하지 않는 것을 확인합니다.
3. **3분:** F5로 실행하고 입력을 바꾼 뒤 실행합니다. 어제와 달리 결과 문구가 갱신되는 것을 확인합니다.
4. **4분:** 창을 닫습니다. Input의 set 안 input=value 줄에 중단점을 걸고 다시 실행합니다. 입력창에 글자 하나를 입력했을 때 멈추는 것을 봅니다. F5로 계속 실행하고 중단점을 제거합니다. 빈 입력을 실행해 안내를 확인한 뒤 종료합니다.

진도가 다르면 저장소 루트의 터미널에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 6로 시작 상태를 적용할 수 있습니다. 파일 복사가 어렵다면 같은 명령의 Stage를 7로 바꾸면 오늘 완료본이 적용됩니다. 실행 중인 앱을 먼저 닫고 Visual Studio에서 변경 파일을 다시 불러옵니다. 기존 교재 파일은 자동 백업됩니다. 단계 적용 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- TwoWay는 값이 오가는 방향, UpdateSourceTrigger는 화면값을 데이터에 쓰는 시점입니다.
- INotifyPropertyChanged는 C# 속성 변경을 화면에 알립니다.
- 바인딩이 있어도 필요한 변경 알림이 없으면 화면에 새 값이 나타나지 않을 수 있습니다.
- 실무·면접에서 자주 묻는 연결은 DataContext → public 속성 → 변경 알림입니다.

완료본: day07. 다음: 2주차 4일차.

2주차 4일차: ObservableCollection과 표 갱신 — 120분

1. 학습 목표 — 5분

- 행 추가 알림과 행 내부 값 변경 알림을 구분합니다.
- 입력값으로 첫 행의 제목을 바꾸고 새 행을 추가합니다.
- 바인딩으로 입력부터 결과 표까지 연결합니다.

시작 상태는 day07입니다. 두 동작은 알림을 관찰하기 위한 학습용이며, 최종 API 프로그램에서는 서버 응답으로 표를 채웁니다.

2. 핵심 개념 설명 — 15분

List<Post>에 처음 들어 있던 두 행은 잘 표시됐습니다. 하지만 화면이 이미 연결된 뒤 목록에 항목을 더하거나 지우는 사실도 알려야 합니다.

ObservableCollection<Post>는 항목 추가·삭제 같은 목록 변화를 알리는 컬렉션입니다. Posts.Add를 호출하면 DataGrid에 새 행이 나타납니다.

컬렉션이 알리는 것과 항목이 알리는 것은 다릅니다. 기존 Post의 Title 값이 바뀌는 것은 목록에 행을 추가한 일이 아닙니다. 그래서 Post도 INotifyPropertyChanged를 구현하여 Title 변경을 알립니다.

바뀌는 것	알림을 맡는 곳
결과·상태 문구	MainWindow의 INotifyPropertyChanged
Posts에 행 추가·삭제	ObservableCollection<Post>
기존 행의 제목	Post의 INotifyPropertyChanged

Posts는 get만 있지만 Posts.Add는 가능합니다. "목록 객체를 다른 것으로 교체"하는 것과 "같은 목록 안에 항목을 추가"하는 것은 다릅니다. 오늘은 목록 객체를 계속 유지합니다.

실무·면접 기초: ObservableCollection을 쓰더라도 항목 속성 변경까지 자동으로 알려 주는 것은 아닙니다. 데이터 변경의 위치를 먼저 구분하세요.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

```
public ObservableCollection<Post> Posts { get; } = new()
{
    new Post { Id = 1, Title = "첫 번째 예제 게시글" },
    new Post { Id = 2, Title = "두 번째 예제 게시글" }
};
```

버튼 처리에서 목록과 항목을 각각 바꿉니다.

```
        Posts[0].Title = text;
        Posts.Add(new Post { Id = Posts.Count + 1, Title = text });
```

Post의 Title에는 이전 시간과 같은 속성 변경 알림을 넣습니다.

```
    private string title = "";
    public string Title
    {
        get => title;
        set
        {
            title = value;
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(nameof(Title)));
        }
    }
```

번호는 학습용으로 현재 개수에 1을 더합니다. 실제 데이터베이스의 기본키 생성 규칙을 구현하는 예제는 아닙니다.

오늘 바뀌는 파일입니다. 링크에는 생략 없는 전체 코드가 있습니다.

파일	전체 코드
Post.cs	day08의 Post.cs
MainWindow.xaml.cs	day08의 MainWindow.xaml.cs

본문에는 핵심 부분만 보여 줍니다. 파일을 만들거나 교체할 때는 위 링크의 **전체 코드**를 사용하세요.
App 파일은 바뀌지 않습니다.

4. Visual Studio에서 따라 하는 실습 — 75분

파일 적용과 실행 — 25분

1. MainWindow.xaml.cs와 Post.cs를 오늘의 전체 코드로 교체합니다. XAML은 day07과 같습니다.
2. MainWindow의 Posts 타입이 ObservableCollection<Post>로 바뀐 것을 봅니다.
3. Post.cs의 Title에 값 저장과 PropertyChanged 호출이 있는지 확인합니다.
4. F5로 실행해 초기 두 행이 보이는지 확인합니다.

두 종류의 변경 관찰 — 25분

1. 입력창에 "변경한 제목"을 넣고 실행합니다. 첫 행 제목이 바뀌고 번호 3인 행이 생깁니다.
2. 입력을 "다음 제목"으로 바꾸고 다시 실행합니다. 첫 행은 새 제목으로 바뀌고 번호 4인 행이 생깁니다.
3. 빈 입력을 실행합니다. 행은 추가되지 않고 입력 안내만 나타납니다.
4. 프로그램을 닫고 다시 실행합니다. 저장 기능이 없으므로 초기 두 행으로 돌아옵니다.

알림의 차이를 코드에서 확인 — 15분

1. Run의 Posts[0].Title 대입문과 Post.cs의 Title set 안 title=value 줄에 각각 중단점을 겁니다.

2. 입력 후 실행하고 첫 중단점에서 F5를 누릅니다. Title의 set 안에서 멈추면 F10으로 값 저장과 속성 알림을 봅니다.
3. F5로 계속 실행합니다. Posts.Add는 컬렉션이 추가 알림을 맡으므로 우리가 별도의 Notify(nameof(Posts))를 호출하지 않았다는 점을 확인합니다.
4. 중단점을 제거하고 실행을 종료합니다.

최종 저장 — 10분

1. 다시 실행해 입력 → 클릭 → 첫 행 변경·새 행 추가를 한 번 이어서 확인합니다.
2. 코드와 화면에서 "속성 하나", "목록 전체", "목록 안의 항목"을 각각 가리켜 봅니다. 별도 문제 풀이 파일은 만들지 않습니다.
3. 저장하고 종료합니다. 이 상태가 3주차 시작 코드입니다.

진도가 다르면 저장소 루트의 터미널에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 7로 시작 상태를 적용할 수 있습니다. 파일 복사가 어렵다면 같은 명령의 Stage를 8로 바꾸면 오늘 완료본이 적용됩니다. 실행 중인 앱을 먼저 닫고 Visual Studio에서 변경 파일을 다시 불러옵니다. 기존 교재 파일은 자동 백업됩니다. 단계 적용 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- 화면과 C# 데이터는 DataContext와 Binding으로 연결합니다.
- 변경을 저장하는 것과 변경 알림을 보내는 것은 구분합니다.
- ObservableCollection은 행의 추가·삭제를, Post의 INotifyPropertyChanged는 제목 변경을 알립니다.
- 표는 읽기 전용입니다. 이번 주에는 편집 기능이나 SQL 연결을 추가하지 않습니다.

완료본: day08. 다음: 3주차 1일차.

3주차 1일차: Model·View·ViewModel의 역할 — 30분

1. 학습 목표 — 2분

- 디자인 패턴을 역할을 나누는 방법으로 이해합니다.
- 화면 상태를 MainViewModel 클래스로 옮깁니다.

시작 상태는 2주차 완료본(day08)입니다.

2. 핵심 개념 설명 — 8분

지금 MainWindow에는 화면 초기화, 입력 상태, 결과 상태, 목록, 실행 동작이 모두 들어 있습니다. 코드가 늘면 "화면 배치 수정"과 "데이터 처리 수정"이 같은 창에 모여 찾기 어려워집니다.

디자인 패턴은 이런 반복적인 구조 문제에 사용하는 역할 분담 방법입니다. MVVM은 이름 세 개를 외우기보다 현재 코드가 맡은 일을 나누는 데서 시작합니다.

역할	이번 프로그램	담당
Model	Post.cs	게시글 한 건의 데이터
View	MainWindow.xaml과 화면 관련 Code-behind	사용자에게 보여 주는 화면
ViewModel	MainViewModel.cs	화면에 제공할 입력·결과·상태·목록

오늘은 상태와 변경 알림만 MainViewModel로 옮깁니다. 버튼 처리 동작은 아직 창에 남겨 두어 한 번에 바꾸는 양을 줄입니다. 완전한 MVVM 전환은 이번 주에 단계적으로 마칩니다.

MainWindow의 DataContext를 새 ViewModel 객체로 지정하면 XAML은 그 객체의 Input·Result·Posts를 찾습니다. 속성 이름을 유지하므로 Binding 경로는 거의 그대로입니다. 파일을 나누는 것보다 **데이터를 찾는 기준 객체가 바뀐 것**이 핵심입니다.

작은 화면에서 이벤트 처리기를 쓰는 것도 자연스럽습니다. MVVM은 프로그램의 목적과 크기에 맞춰 선택하는 구조입니다.

3. 짧고 이해하기 쉬운 코드 예제 — 4분

```
    private readonly MainViewModel viewModel = new();

    public MainWindow()
    {
        InitializeComponent();
        DataContext = viewModel;
    }
```

MainViewModel은 Window를 상속하지 않는 일반 C# 클래스입니다. Input·Result·Status·Posts와 변경 알림이 그 안으로 이동합니다. 오늘은 창에서 결과를 쓸 수 있도록 Result와 Status의 set을 public으로 둡니

다.

버튼 처리기의 일부는 다음처럼 바뀝니다.

```
        viewModel.Result = text;
        viewModel.Status = "첫 행의 제목을 수정하고 새 행을 추가했습니다.";
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainViewModel.cs	day09의 MainViewModel.cs
MainWindow.xaml.cs	day09의 MainWindow.xaml.cs

4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** day08이 실행되는지 확인하고 창을 닫습니다.
2. **5분:** 프로젝트에 MainViewModel.cs를 추가하고 전체 코드를 넣습니다. MainWindow.xaml.cs를 오늘의 전체 코드로 교체합니다. XAML과 Post.cs는 그대로입니다.
3. **3분:** F5로 실행해 입력 후 버튼을 누릅니다. 첫 행 변경과 새 행 추가가 이전과 똑같이 동작하는지 확인합니다.
4. **3분:** 창을 닫고 MainWindow와 MainViewModel을 나란히 엽니다. 속성·알림은 ViewModel에, 버튼 처리 순서는 창에 있다는 경계를 확인합니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 8로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 9로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- Model은 데이터, View는 화면, ViewModel은 화면에 제공할 상태를 맡습니다.
- MainViewModel은 일반 C# 객체이며 화면 컨트롤을 상속하지 않습니다.
- DataContext를 새 객체로 바꾸어도 같은 public 속성이 있으면 기존 바인딩을 이어 쓸 수 있습니다.
- 오늘은 전환 중간 단계입니다. 역할을 한 번에 전부 옮기지 않았습니다.

완료본: day09. 다음: 3주차 2일차.

3주차 2일차: ViewModel로 실행 동작 옮기기 — 30분

1. 학습 목표 — 2분

- 입력 확인과 목록 변경을 ViewModel의 메서드로 옮깁니다.
- 화면의 이벤트가 데이터 처리 메서드를 호출하는 관계를 이해합니다.

시작 상태는 day09입니다.

2. 핵심 개념 설명 — 6분

어제는 데이터만 옮겼습니다. 오늘은 "입력을 읽고 확인한 뒤 목록을 바꾸는 일"도 MainViewModel.Run으로 옮깁니다. 이 동작은 어떤 TextBox에 입력되었는지 몰라도 Input 문자열만 있으면 수행할 수 있기 때문입니다.

Code-behind의 이벤트 처리기는 viewModel.Run을 호출하는 한 줄만 남습니다. ViewModel은 InputTextBox나 PostsGrid 같은 화면 요소 이름을 참조하지 않습니다.

```
입력창 —— TwoWay Binding —— Input
버튼 Click → 창의 처리기 → ViewModel.Run()
ViewModel의 Result·Status·Posts 변경 → 알림 → 화면 갱신
```

Result와 Status를 ViewModel 안에서만 바꾸므로 set을 private으로 좁힙니다. 화면의 바인딩은 get으로 읽기만 하므로 계속 동작합니다.

DataContext는 ViewModel 자체가 아니라, View가 "어느 객체를 읽을지" 지정하는 연결 지점입니다. 앞으로는 초기화 때 한 번 지정하고 유지합니다. 오늘 버튼은 여전히 이벤트를 사용합니다. Command는 다음 시간에 연결합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    private readonly MainViewModel viewModel = new();

    public MainWindow()
    {
        InitializeComponent();
        DataContext = viewModel;
    }

    private void RunButton_Click(object sender, RoutedEventArgs e)
    {
        viewModel.Run();
    }
```

```
}
```

MainViewModel의 Run 안에는 이전 시간 창에 있던 문자열 확인, 첫 행 제목 변경, 새 행 추가가 그대로 들어갑니다. 동일한 동작의 위치만 바꿉니다.

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainViewModel.cs	day10의 MainViewModel.cs
MainWindow.xaml.cs	day10의 MainWindow.xaml.cs

4. Visual Studio에서 따라 하는 실습 — 14분

1. **2분:** 실행을 종료합니다. 오늘의 MainWindow.xaml.cs와 MainViewModel.cs를 각각 전체 코드로 교체합니다.
2. **4분:** 창의 처리기가 viewModel.Run만 호출하는지 확인합니다. ViewModel에서 Input·Result·Status·Posts 이름을 찾습니다.
3. **4분:** F5로 실행하여 정상 입력과 빈 입력을 확인합니다. 기존 동작이 유지되어야 합니다.
4. **4분:** Run 메서드 첫 줄에 중단점을 걸고 버튼을 누릅니다. 창의 이벤트에서 ViewModel의 메서드로 넘어왔는지 봅니다. 중단점을 제거하고 종료합니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 9로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 10로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- ViewModel은 화면 컨트롤이 아니라 데이터 속성을 사용합니다.
- 처리 순서가 바뀌지 않아도 코드의 책임을 옮길 수 있습니다.
- Code-behind는 이벤트를 전달하고, ViewModel은 상태와 동작을 관리합니다.
- 실무·면접 기초: MVVM은 폴더 세 개를 만드는 규칙보다 역할과 의존 관계를 나누는 방법입니다.

완료본: day10. 다음: 3주차 3일차.

3주차 3일차: ICommand와 RelayCommand — 30분

1. 학습 목표 — 2분

- Command가 실행 동작과 실행 가능 여부를 제공함을 이해합니다.
- CommunityToolkit.Mvvm으로 버튼을 ViewModel의 명령에 연결합니다.

시작 상태는 day10입니다. 이번 시간부터 외부 NuGet 패키지를 사용합니다. 최초 다운로드 대기는 학습 시간에서 제외합니다.

2. 핵심 개념 설명 — 8분

이벤트 방식에서는 버튼이 창의 처리기를 호출했습니다. Command 방식에서는 버튼이 바인딩으로 명령 객체를 찾아 실행을 요청합니다.

ICommand는 Execute(실행), CanExecute(실행 가능 여부), CanExecuteChanged(가능 여부를 다시 확인하라는 알림)를 약속하는 인터페이스입니다. 직접 구현하면 반복 코드가 생기므로 CommunityToolkit.Mvvm의 RelayCommand를 사용합니다. 공식 RelayCommand 안내

이번 코드	의미
new RelayCommand(Run, CanRun)	실행할 메서드와 가능 여부 판단 메서드를 명령으로 묶음
Command="{Binding RunCommand}"	버튼이 ViewModel의 명령을 사용
NotifyCanExecuteChanged()	입력이 바뀌었으니 버튼의 가능 여부 재확인

CanRun은 공백이 아닌 입력이 있을 때 true입니다. Input이 바뀌면 명령에 알림을 보내 빈 입력에서는 버튼이 비활성화됩니다. RelayCommand가 Input 변화를 저절로 알아채는 것은 아닙니다.

Run 뒤에 괄호를 붙이지 않은 이유는 지금 실행하지 않고 "나중에 실행할 메서드"를 전달하기 때문입니다. 람다식과 델리게이트 이론은 깊게 다루지 않습니다.

교재는 NuGet에서 확인한 CommunityToolkit.Mvvm 8.4.2로 버전을 고정합니다. 최신 버전으로 계속 바꾸는 학습은 하지 않습니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
public IRelayCommand RunCommand { get; }

public MainViewModel()
{
    RunCommand = new RelayCommand(Run, CanRun);
}

private bool CanRun()
{
    return !string.IsNullOrWhiteSpace(Input);
}
```

Input의 set에 들어 있는 알림입니다.

```
        input = value;
        Notify(nameof(Input));
        RunCommand.NotifyCanExecuteChanged();
```

버튼의 Click 연결은 제거하고 Command를 연결합니다.

```
<Button Command="{Binding RunCommand}" Content="실행" />
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
WpfTestApp.csproj	day11의 WpfTestApp.csproj
MainWindow.xaml	day11의 MainWindow.xaml
MainViewModel.cs	day11의 MainViewModel.cs
MainWindow.xaml.cs	day11의 MainWindow.xaml.cs

4. Visual Studio에서 따라 하는 실습 — 12분

1. **3분:** 실행을 종료합니다. 프로젝트 우클릭 → **NuGet 패키지 관리** → 찾아보기에서 CommunityToolkit.Mvvm을 검색합니다. 버전 8.4.2를 선택하여 설치합니다. 설치 후 csproj에 PackageReference가 있는지 봅니다. 전체 csproj를 교체한 경우에는 솔루션 빌드 시 복원되므로 중복 설치할 필요가 없습니다.
2. **4분:** MainWindow 두 파일과 MainViewModel.cs를 오늘의 전체 코드로 교체합니다. using CommunityToolkit.Mvvm.Input이 추가되고, 창의 클릭 처리기가 제거된 것을 확인합니다. Post.cs는 유지합니다.
3. **3분:** F5로 실행합니다. 기본 입력에서는 버튼이 활성화되고, 모두 지우면 비활성화됩니다. 다시 텍스트를 입력하고 실행해 기존 행 변경이 되는지 확인합니다.
4. **2분:** 입력값 변경 → Input의 set → NotifyCanExecuteChanged → 버튼 상태 재확인 경로를 코드에서 찾고 종료합니다.

패키지를 찾지 못하면 NuGet 원본이 nuget.org인지와 인터넷 연결을 확인합니다. IRelayCommand는 Toolkit이 제공하는 ICommand 확장 인터페이스이며 내부 구현을 암기하지 않습니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 10로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 11로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- Command는 동작과 실행 가능 여부를 화면에 제공합니다.

- RelayCommand는 ICommand 구현의 반복 작업을 줄입니다.
- 입력 변경 시 NotifyCanExecuteChanged를 호출해야 버튼이 다시 판단합니다.
- 클릭을 Command로 연결한 뒤 ViewModel은 버튼을 직접 수정하지 않습니다.

완료본: day11. 다음: 3주차 4일차.

3주차 4일차: Toolkit으로 MVVM 기본 구조 완성 — 120분

1. 학습 목표 — 5분

- 반복되는 속성 알림을 ObservableObject와 SetProperty로 줄입니다.
- 이벤트 방식과 MVVM 방식의 차이를 같은 화면에서 정리합니다.
- 다음 주 API 결과를 받을 작은 MVVM 프로그램을 준비합니다.

시작 상태는 day11입니다.

2. 핵심 개념 설명 — 15분

지금 MainViewModel과 Post에는 PropertyChanged 이벤트를 선언하고 호출하는 코드가 반복됩니다. ObservableObject는 이 알림 기능을 제공하는 Toolkit의 기반 클래스입니다. SetProperty는 값이 달라졌는지 확인하고, 필드에 저장하고, 속성 변경을 알립니다. 공식 ObservableObject 안내

set => SetProperty(ref title, value)는 짧게 쓴 속성 설정 코드입니다. ref는 SetProperty가 실제 title 필드를 바꿀 수 있게 전달하는 C# 표기입니다. Input은 값이 달라진 경우 명령 상태도 재확인해야 하므로 if 문으로 SetProperty의 반환값을 사용합니다.

1주차 방식	오늘 방식
창의 Click 처리기	ViewModel의 RunCommand
InputTextBox.Text 읽기	Input 속성 읽기
ResultTextBlock.Text 쓰기	Result 변경과 속성 알림
화면 컨트롤을 직접 조작	데이터·명령을 Binding으로 연결

이번 완료본에서는 2주차의 "첫 행 수정 + 행 추가" 관찰용 동작을 정리합니다. 실행할 때 기존 목록을 지우고 입력 텍스트 한 건을 결과 표에 넣습니다. 다음 주에는 이 자리에 API 결과가 들어갑니다. 버튼을 추가하지 않습니다.

소스 생성기 특성, DI, 메시징, 서비스 계층은 사용하지 않습니다. 현재 Code-behind에 InitializeComponent와 DataContext 설정이 남아 있어도 역할에 맞는 코드입니다.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

```
    private string input = "안녕하세요, WPF";
    public string Input
    {
        get => input;
        set
        {
            if (SetProperty(ref input, value))
            {
                RunCommand.NotifyCanExecuteChanged();
            }
        }
    }
```

Result와 Status도 SetProperty를 사용합니다. 직접 선언했던 event와 Notify 메서드는 제거합니다. 실행의 핵심은 다음과 같습니다.

```
        Posts.Clear();
        Posts.Add(new Post { Id = 1, Title = text });
```

창의 C# 파일은 그대로 두고, ViewModel과 Post의 알림 코드를 간단하게 바꿉니다.

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
Post.cs	day12의 Post.cs
MainViewModel.cs	day12의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 75분

코드 전환 — 25분

1. 창을 닫고 MainViewModel.cs와 Post.cs를 오늘의 전체 코드로 교체합니다.
2. 두 클래스가 ObservableObject를 상속하는지 확인합니다.
3. Result·Status·Title의 set에 SetProperty가 있는지 봅니다. MainViewModel에서 직접 선언한 PropertyChanged와 Notify가 없어졌는지도 확인합니다.
4. 솔루션을 빌드합니다. Toolkit 관련 형식을 찾지 못하면 8.4.2 패키지 복원 상태를 확인합니다.

같은 화면으로 동작 확인 — 20분

1. F5로 실행합니다. 이번 완료본은 결과 목록이 비어 있는 상태로 시작합니다.
2. "MVVM 테스트"를 입력하고 실행하면 번호 1의 한 행이 나타납니다.
3. 다른 입력을 실행하면 이전 행 대신 새 텍스트 한 건이 나타납니다.
4. 입력을 지우면 버튼이 비활성화되고, 다시 넣으면 활성화되는지 봅니다.

데이터가 흐르는 순서 관찰 — 20분

1. Input의 set에 중단점을 걸고 입력합니다. 화면값이 ViewModel로 들어오는 것을 확인합니다.
2. 중단점을 옮겨 Run의 Result 대입문에서 멈춥니다. F10으로 값을 바꾸면 속성 알림을 통해 화면이 갱신됩니다.
3. MainWindow.xaml에서 Input·RunCommand·Posts·Result·Status 바인딩을 차례로 찾습니다.
4. 중단점을 모두 제거하고 실행을 종료합니다.

다음 주를 위한 저장 — 10분

1. 다시 실행해 입력·명령·결과 표의 연결을 확인합니다.
2. 파일을 역할로 구분해 봅니다. Post는 데이터, XAML은 화면, MainViewModel은 화면에 제공할 상태와 동작입니다.
3. 저장하고 종료합니다. 새 프로젝트는 만들지 않습니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 11로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 12로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- ObservableObject와 SetProperty는 지난주 배운 변경 알림의 반복 구현을 대신합니다.
- Toolkit을 써도 DataContext·Binding·속성 알림의 원리는 같습니다.
- ViewModel에는 화면 컨트롤을 직접 참조하는 코드가 없습니다.
- 구조를 복잡하게 확장하지 않고, 다음 주에는 Run만 비동기 요청으로 바꿉니다.

완료본: day12. 다음: 4주차 1일차.

4주차 1일차: async/await로 기다리는 동안 화면 유지 — 30분

1. 학습 목표 — 2분

- 기다리는 작업이 화면을 멈추지 않도록 await를 사용합니다.
- Task를 반환하는 메서드를 AsyncRelayCommand에 연결합니다.

시작 상태는 3주차 완료본(day12)입니다. 오늘은 인터넷을 사용하지 않고 2초 대기만 흉내 냅니다.

2. 핵심 개념 설명 — 7분

Windows 화면은 보통 UI 스레드에서 입력과 그리기를 처리합니다. 이 스레드를 긴 대기로 붙잡으면 창 이동이나 입력도 멈춥니다.

Task는 진행 중인 작업을 나타냅니다. async 메서드 안에서 아직 끝나지 않은 Task를 await하면 그 메서드는 일단 제어를 돌려주고, 작업이 끝난 뒤 다음 줄을 이어서 실행합니다. await 자체가 새 스레드를 무조건 만드는 것은 아닙니다. C# 비동기 공식 안내

코드	역할
async Task RunAsync()	기다릴 수 있는 작업을 반환하는 메서드
await Task.Delay(2000)	UI 스레드를 붙잡지 않고 2초 대기
new AsyncRelayCommand(RunAsync, CanRun)	Task 반환 메서드를 버튼의 명령으로 제공

AsyncRelayCommand는 기본 설정에서 실행 중에는 CanExecute를 false로 알려 버튼 중복 실행을 막습니다. 이번에는 별도의 실행 중 플래그나 취소 버튼을 만들지 않습니다. 공식 AsyncRelayCommand 안내

.Result나 .Wait()로 강제로 기다리거나 Thread.Sleep으로 UI 스레드를 멈추지 않습니다. 오늘은 RunAsync가 시작할 때 입력값을 지역 변수에 복사하므로, 대기 중 입력창을 수정해도 결과는 클릭 당시 입력을 사용합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
public IAsyncRelayCommand RunCommand { get; }

public MainViewModel()
{
    RunCommand = new AsyncRelayCommand(RunAsync, CanRun);
}
```

처리 메서드는 다음과 같습니다.

```
    private async Task RunAsync()
    {
        string text = Input.Trim();
        Posts.Clear();
        Result = "처리 중입니다.";
        Status = "2초를 기다리는 동안에도 창을 움직일 수 있습니다.";

        await Task.Delay(2000);

        Posts.Add(new Post { Id = 1, Title = text });
        Result = "1건을 표시했습니다.";
        Status = "대기가 끝났습니다.";
    }
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainWindow.xaml	day13의 MainWindow.xaml
MainViewModel.cs	day13의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** 실행을 종료하고 MainViewModel.cs와 MainWindow.xaml을 오늘의 전체 코드로 교체합니다. 제목만 바뀌며 화면 배치와 Command 바인딩은 같습니다.
2. **4분:** Run이 RunAsync로 바뀌고, 반환형은 void 대신 Task가 된 것을 확인합니다. C# 파일에는 async void 메서드를 추가하지 않습니다.
3. **4분:** F5로 실행하고 입력 후 버튼을 누릅니다. 2초 동안 상태 문구가 보이고 버튼이 비활성화됩니다. 그동안 창을 움직이고 입력창의 텍스트를 바꿔 봅니다.
4. **3분:** 결과가 클릭 당시 텍스트로 표시되는지 확인합니다. 대기가 끝나면 버튼이 다시 활성화됩니다. 창을 닫습니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 12로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 13로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- await는 비동기 작업이 끝날 때까지 기다린 뒤 다음 줄을 실행합니다.
- 기다리는 동안 UI 스레드가 다른 화면 작업을 처리할 수 있습니다.
- AsyncRelayCommand는 Task 반환 메서드를 연결하고 실행 중 버튼 상태를 관리합니다.
- 실무·면접 기초: async/await와 "새 스레드 생성"을 같은 의미로 설명하지 않습니다.

완료본: day13. 다음: 4주차 2일차.

4주차 2일차: HttpClient로 고정 URL GET 호출 — 30분

1. 학습 목표 — 2분

- REST API의 GET 응답을 C# 객체 목록으로 읽습니다.
- 고정된 게시글 URL의 응답을 DataGrid에 표시합니다.

시작 상태는 day13입니다. 이 시간부터 인터넷 연결이 필요합니다.

2. 핵심 개념 설명 — 7분

GET은 지정한 주소에서 데이터를 조회하는 HTTP 요청입니다. 이번에는 JSONPlaceholder의 학습용 게시글 API를 사용합니다. 공식 API 안내에서 계정이나 API 키 없이 사용할 수 있는 예제로 제공하며, 게시글 목록 응답은 userId·id·title·body가 있는 객체의 배열입니다.

JSON은 데이터를 텍스트로 표현하는 형식입니다. 바깥 대괄호는 여러 항목, 각 중괄호는 게시글 하나를 나타냅니다. Post 클래스가 항목 모양, List<Post>가 응답 전체 모양에 대응합니다. 예를 들어 소문자 title은 Post.Title로 읽어 옵니다.

GetFromJsonAsync<List<Post>>는 GET 요청, 성공 상태 확인, JSON을 목록으로 바꾸는 작업을 도와줍니다. System.Net.Http.Json의 확장 메서드이며 .NET 10에 포함되어 있으므로 별도 JSON 패키지를 설치하지 않습니다. 이 메서드의 웹 기본 설정에서는 속성 이름의 대소문자를 구분하지 않고 매핑합니다. 공식 JSON 확장 메서드 안내

HttpClient는 요청마다 새로 만들지 않고 static readonly 필드 한 개를 재사용합니다. 이번 작은 프로그램에서는 15초의 요청 제한 시간만 설정합니다. HttpClient 공식 사용 지침

네트워크는 실패할 수 있어 전체 코드에 기본 catch를 미리 제공합니다. 오늘은 요청과 데이터 변환에 집중하고 오류별 처리는 다음 시간에 확인합니다. API 작업에서 발생한 예외를 AsyncRelayCommand가 자동으로 상태 문구로 바꿔 주는 것은 아닙니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
namespace WpfTestApp;

// JSON 응답을 이 C# 클래스의 속성으로 읽어 옵니다.
public class Post
{
    public int UserId { get; set; }
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public string Body { get; set; } = "";
}
```

ViewModel의 HttpClient 필드와 요청 부분입니다.

```
    private static readonly HttpClient client = new()
    {
        Timeout = TimeSpan.FromSeconds(15)
    };
```

```
            List<Post>? posts = await client.GetFromJsonAsync<List<Post>>(
                "https://jsonplaceholder.typicode.com/posts?userId=1");
```

오늘은 요청 URL이 코드에 고정되어 있습니다. 입력창을 바꾸어도 아직 요청 대상은 바뀌지 않습니다. 화면 안내에도 이 점을 표시합니다. Task.Delay는 제거하고 실제 요청을 await합니다. JSON을 읽은 후 Posts.Add로 채우는 부분은 이미 배운 컬렉션 갱신입니다.

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainWindow.xaml	day14의 MainWindow.xaml
Post.cs	day14의 Post.cs
MainViewModel.cs	day14의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** 실행을 종료하고 오늘 바뀌는 세 파일을 전체 코드로 교체합니다. Post는 API에서 읽을 네 속성을 가진 단순 클래스로 바뀝니다. API 행은 편집하지 않으므로 Title 변경 알림은 더 이상 넣지 않습니다.
2. **3분:** MainViewModel의 using System.Net.Http.Json과 static HttpClient, GetFromJsonAsync 호출을 찾습니다.
3. **4분:** F5로 실행하고 실행 버튼을 누릅니다. 결과 표에 게시글이 나타나는지 확인합니다. 조회 시점의 정상 응답은 10건이며 제목은 API가 제공하는 원문입니다.
4. **4분:** 입력창 끝의 userId=1을 userId=2로 바꾸고 다시 실행합니다. 아직 고정 주소를 쓰므로 같은 첫 데이터가 나오는 것을 확인합니다. 창을 닫습니다. 오류 상태가 보이면 인터넷 연결과 공식 URL 접근 여부를 확인합니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 13로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 14로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- GET으로 받은 JSON 배열을 List<Post>로 읽을 수 있습니다.
- HttpClient는 한 번 만들고 재사용합니다.
- 서버에서 받은 목록을 ObservableCollection에 넣으면 표가 갱신됩니다.
- 요청 주소는 아직 고정이며 다음 시간에 입력창과 연결합니다.

완료본: day14. 다음: 4주차 3일차.

4주차 3일차: 입력 URL과 기본 오류 처리 — 30분

1. 학습 목표 — 2분

- 입력한 URL을 실제 GET 요청에 사용합니다.
- 잘못된 주소, HTTP 실패, JSON 형식 오류, 시간 초과를 상태 문구로 안내합니다.

시작 상태는 day14입니다.

2. 핵심 개념 설명 — 6분

URL을 입력받으면 잘못된 형식이 들어올 수 있습니다. Uri.TryCreate는 문자열이 절대 주소로 해석되는지 확인합니다. 그다음 http 또는 https인지 확인합니다. 이는 주소 형식 확인이며 서버가 존재하거나 응답 형식이 맞는지까지 보장하지 않습니다.

성공하면 생성한 uri를 GetFromJsonAsync에 넘깁니다. 호출을 시작할 때 기존 행을 비워 실패 후에도 이전 결과가 새 결과처럼 보이지 않게 합니다. 0건을 나타내는 빈 배열 []는 정상 결과이고, JSON null이나 객체 하나는 이 프로그램이 기대한 배열이 아닙니다.

상황	처리
"잘못된주소"나 file 주소	요청 전에 상태 안내 후 return
연결 실패 또는 HTTP 실패 상태	HttpRequestException 처리
JSON 문법·형식 불일치	JsonException 처리
15초 제한 초과	OperationCanceledException 처리

AsyncRelayCommand의 기본 설정은 예외를 조용히 삼키지 않습니다. 예상 가능한 요청 오류는 RunAsync 안에서 catch합니다. 여기에는 취소 버튼이 없으므로 OperationCanceledException은 시간 초과 안내로 사용합니다.

WPF 버튼에서 호출된 이 메서드는 기본 await 동작으로 UI 컨텍스트에 이어집니다. 따라서 await 뒤에 Posts를 채우는 데 별도 Task.Run이나 Dispatcher 코드를 넣지 않습니다. UI 스레드와 다른 스레드에서 컬렉션을 수정하는 고급 주제는 제외합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
        if (!Uri.TryCreate(Input.Trim(), UriKind.Absolute, out Uri? uri)
            || (uri.Scheme != Uri.UriSchemeHttp && uri.Scheme != Uri.UriSchemeHttps))
        {
            Status = "http 또는 https로 시작하는 API URL을 입력해 주세요.";
            return;
        }
```

요청 부분은 고정 문자열 대신 uri를 사용합니다.

```
        List<Post>? posts = await client.GetFromJsonAsync<List<Post>>(
            uri);
```

기본 오류 처리 중 HTTP 실패 안내는 다음과 같습니다. 전체 파일에는 JSON 오류와 시간 초과 catch도 이어집니다.

```
    catch (HttpRequestException)
    {
        Status = "요청에 실패했습니다. URL, 인터넷 연결, HTTP 응답 상태를 확인하세요.";
    }
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainWindow.xaml	day15의 MainWindow.xaml
MainViewModel.cs	day15의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 14분

1. **3분:** 실행을 종료하고 MainViewModel.cs와 MainWindow.xaml을 전체 코드로 교체합니다. 입력창 위 안내가 실제 URL 입력으로 바뀌었는지 확인합니다.
2. **4분:** F5로 실행합니다. 기본 주소로 조회한 후 userId=1을 userId=2로 바꾸고 조회합니다. 다른 작성자 데이터가 나타나는지 확인합니다.
3. **3분:** "잘못된주소"를 입력하고 실행합니다. 요청 전에 안내가 표시되고 기존 표는 비워집니다. 빈 입력에서는 버튼이 비활성화되는 것도 확인합니다.
4. **4분:** 주소를 https://jsonplaceholder.typicode.com/posts/1 로 바꾸어 실행합니다. 이는 단일 객체이므로 배열 형식 오류 안내가 나타납니다. 기본 주소로 복구해 다시 성공하는지 확인하고 종료합니다.

시간 초과는 네트워크 환경에 따라 재현 시간이 달라 일부러 만들지 않아도 됩니다. 코드에서 15초 설정과 해당 catch의 위치만 확인합니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 14로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 15로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- 입력 URL의 형식 확인과 실제 요청 성공은 별개의 문제입니다.
- 예상 가능한 오류는 catch하여 상태 문구로 알립니다.
- 실패 후에는 이전 결과를 표시하지 않고, 이후 정상 요청을 다시 실행할 수 있습니다.
- API 응답 모양에 맞는 C# 클래스가 필요하며 임의 JSON을 자동으로 표로 바꾸지는 않습니다.

완료본: day15. 다음: 4주차 4일차.

4주차 4일차: 작은 WPF API 테스트 프로그램 완성 — 120분

1. 학습 목표 — 5분

- 입력 URL → Command → 비동기 GET → 데이터 변환 → 표 표시를 연결합니다.
- 응답 항목의 번호와 제목을 최소한으로 확인합니다.
- 새 기능을 늘리지 않고 최종 프로그램을 완성합니다.

시작 상태는 day15입니다. 최종 화면은 입력창 하나, 실행 버튼 하나, 결과 표, 결과 건수와 상태 문구입니다.

2. 핵심 개념 설명 — 15분

지난 시간의 코드는 JSON 배열이면 Post로 읽어 옵니다. 그런데 전혀 다른 데이터의 배열인 [{}]도 기본값이 들어 있는 Post가 될 수 있습니다. 형식을 객체 목록으로 읽었다는 사실만으로 학습용 게시글 데이터라고 보장되지는 않습니다.

최종 코드에는 각 항목의 Id가 양수이고 Title이 비어 있지 않은지 확인하는 작은 조건문을 추가합니다. 하나라도 맞지 않으면 표에 넣기 전에 JsonException으로 처리합니다. 이는 이 교재의 게시글 데이터 계약에 맞춘 최소 확인이며 모든 API의 공통 검증 규칙은 아닙니다. UserId와 Body는 JSON 매핑을 이해하기 위해 보관하지만 화면에는 표시하지 않습니다.

정상 응답이 빈 배열이면 반복문을 건너뛰어 결과 0건을 표시합니다. null은 배열이 아니므로 형식 오류로 처리합니다. 항목 검사를 먼저 마친 뒤 행을 추가하므로 검사 실패 시 일부 행만 남지 않습니다.

```
TextBox → Input
Button → RunCommand → RunAsync
URL 확인 → HttpClient GET → List<Post>
항목 확인 → ObservableCollection<Post> → DataGrid
오류 발생 → catch → Status → 화면 안내
```

입력창과 결과를 잇는 Binding, DataContext에 연결한 ViewModel, 속성과 목록의 변경 알림, 비동기 요청이 처음부터 끝까지 이어졌습니다. 1주차와 비교해도 화면 구성은 간단하게 유지합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

```
        // 항목에 번호와 제목이 있는지 최소한으로 확인합니다.
        foreach (Post post in posts)
        {
            if (post is null || post.Id <= 0 || string.IsNullOrWhiteSpace(post.Title))
            {
                throw new JsonException("번호와 제목이 있는 게시글 배열이 필요합니다.");
            }
        }
```

이 검사 다음에 목록을 채우고 건수를 표시합니다.

```
        foreach (Post post in posts)
        {
            Posts.Add(post);
        }

        Result = $"결과 {Posts.Count}건";
        Status = "GET 요청을 완료했습니다.";
```

전체 파일은 MainViewModel.cs 링크에 있습니다. 오늘 추가하는 기능은 이 검사뿐이며 새 화면이나 패키지는 없습니다.

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainViewModel.cs	day16의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 75분

최종 코드 적용 — 20분

1. 창을 닫고 MainViewModel.cs를 오늘의 전체 코드로 교체합니다. 다른 파일은 day15와 같습니다.
2. 요청 → null 확인 → 항목 확인 → 행 추가 → 성공 안내의 순서를 찾습니다.
3. 솔루션을 빌드하고 F5로 실행합니다. 기본 URL로 결과가 표시되는지 확인합니다.

정상 결과와 빈 결과 — 20분

1. https://jsonplaceholder.typicode.com/posts?userId=1 로 조회합니다.
2. userId=2로 바꾸어 다른 결과가 표시되는지 확인합니다.
3. userId=999999로 바꾸어 조회합니다. 현재 학습용 API에서는 []를 반환하므로 "결과 0건"과 완료 상태가 표시됩니다.
4. 기본 URL로 되돌립니다. 실제 서버 데이터나 제공 정책이 바뀌면 공식 API 안내를 기준으로 확인합니다.

오류 후 다시 실행 — 15분

1. "잘못된주소"를 실행하여 URL 안내를 확인합니다.
2. https://jsonplaceholder.typicode.com/posts/1 로 실행하여 단일 객체 형식 오류를 확인합니다.
3. 기본 URL로 다시 조회해 정상적으로 사용할 수 있는지 확인합니다.
4. API 응답이 빠르면 비활성화가 잠깐만 보일 수 있습니다. 비동기 대기는 day13의 2초 예제에서 이미 확인했습니다. 별도 인위적 지연은 최종 코드에 넣지 않습니다.

전체 흐름 확인과 저장 — 20분

1. XAML의 Input, RunCommand, Posts, Result, Status를 찾습니다.

2. MainWindow.xaml.cs의 DataContext=new MainViewModel()을 찾습니다.
3. MainViewModel에서 입력 URL을 사용하는 줄, await 줄, 결과 목록을 채우는 줄을 차례로 봅니다.
4. 중단점이 남았다면 제거하고 저장합니다.
5. 창을 닫았다 다시 실행해 기본 URL로 한 번 조회합니다. 결과를 파일이나 데이터베이스에 저장하지 않으므로 이전 조회 결과는 남지 않습니다.

별도의 과제나 배포 절차는 없습니다. 이 프로그램을 작은 테스트 창을 만들어 본 학습 경험으로 남기면 됩니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 15로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 16로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- XAML은 화면을, ViewModel은 화면 상태와 동작을, Post는 응답 데이터 모양을 맡습니다.
- Binding과 알림으로 C# 데이터 변경이 화면에 반영됩니다.
- Command로 실행을 연결하고 await로 요청 대기 중 화면 반응을 유지합니다.
- 결과 표는 예상한 게시글 배열을 위한 것이며 다른 API에는 모델과 매핑 수정이 필요합니다.
- 실무·면접에서는 직접 만든 이 흐름을 설명할 수 있으면 됩니다. 고급 설계나 WPF 전체 기능을 암기할 필요는 없습니다.

완료본: day16. 전체 학습 안내로 돌아가기.